

The platform, *measured.*

A technical reference for the RigaCap signal platform — the infrastructure it runs on, the data it ingests, the services that turn that data into two tiers of signal, and the production discipline that keeps live output in lockstep with the walk-forward record.

SECTIONS

12

Full audit

ENDPOINTS

120+

FastAPI
surface

SYMBOLS
SCANNED

~4,000

Daily, US
equities

DB TABLES

18

PostgreSQL 15

What this document *covers*.

A complete technical audit of the RigaCap platform: the infrastructure it runs on, the market data it ingests, the services that turn that data into a two-tier signal product, and the operational discipline that keeps production output faithful to the walk-forward record it is marketed on.

INFRASTRUCTURE

1. System Overview	03
2. AWS Infrastructure	04
3. CDN, DNS & SSL	06

DATA LAYER

4. Database Schema	07
5. Data Pipeline & Caching	10

APPLICATION

6. Backend Services & Tier Serving	12
7. API Surface (120+ endpoints)	14
8. Frontend Architecture	16
9. Social & Content Engine	17

OPERATIONS

10. Security Architecture	18
---------------------------	----

11. CI/CD, DR & Deploy Safety	19
12. Monitoring, Performance & Dependencies	20

A serverless platform with a *scheduled* heartbeat.

RigaCap is a serverless, event-driven signal platform on AWS. It ingests daily market data for roughly four thousand US equities through a dual-source pipeline, ranks that universe through a regime-adaptive momentum engine, and delivers the result as two subscriber tiers — Preserver and Maximizer — through a React dashboard, an automated email pipeline, a weekly newsletter, and a moderated social engine.

The backend runs on two specialized AWS Lambda functions — an API Lambda for HTTP traffic and a Worker Lambda for scheduled and long-running jobs — both built from a single container image and differentiated by an environment variable. FastAPI sits on top of Mangum for ASGI compatibility. PostgreSQL 15 on RDS is the system of record. S3 holds the historical price cache and the pre-computed dashboard JSON that CloudFront serves to subscribers; an ElastiCache Redis node fronts hot read paths.

Two-Lambda split. The API Lambda is small and fast (1 GB, 30s timeout) because subscribers expect dashboard loads under two seconds — it never loads the price store and reads only S3 JSON and the database. The Worker Lambda is large and patient (4 GB, 900s timeout) because daily scans, walk-forward simulations, and email runs need both memory and time. One image, two roles, atomic deploys.

Request and event flow

```
# Subscriber request path
React 18 SPA → CloudFront → WAF → API Gateway → API Lambda
                                                    → RDS / Redis /

# Scheduled / event-driven path
EventBridge → Worker Lambda → RDS / S3 (read+write)
                                → Step Functions (walk-forward fan-out)
                                → Claude API (content, autopsies, optimization)
                                → SES (email)
                                → X / Threads / Instagram (social)
```

What the platform produces

- Two tiers of daily signal. A single scan produces both the Preserver (protect) and Maximizer (grow) reads, cached as JSON on S3 and fronted by CloudFront for sub-second dashboard loads.
- Daily digest email. Per-subscriber summary of signals, watchlist, regime context, and any sell alerts — gated on a same-day data-freshness check.
- Weekly newsletter. A market-commentary issue, locked as a draft on Saturday and published Sunday morning.
- Social posts. AI-drafted from real walk-forward and live results, gated by one-click admin approval, auto-published Monday / Wednesday / Friday across X, Threads, and Instagram.
- Mobile push notifications. Sell alerts and high-conviction signal events through Expo.

Signals only. RigaCap publishes signals; subscribers execute at their own broker. The platform never touches customer funds and holds no brokerage integration — a deliberate architectural boundary that keeps the system a publisher, not a manager (see §12 disclosures).

Terraform, two Lambdas, and a *quiet* RDS.

All infrastructure is managed through Terraform

(`infrastructure/terraform/main.tf`) against AWS account `149218244179` in `us-east-1`. State lives in a remote S3 backend with DynamoDB locking, so plans never collide between CI and the operator's laptop. The live Lambda functions carry a deliberate `lifecycle { ignore_changes }` guard on their image and environment (see §11) so a routine `apply` can never clobber production secrets.

Compute

RESOURCE	IDENTIFIER	CONFIGURATION
API Lambda	<code>rigacap-prod-api</code>	Container (ECR), 1024 MB, 30s timeout, <code>LAMBDA_ROLE=api</code> . HTTP requests only; skips the price-store load entirely.
Worker Lambda	<code>rigacap-prod-worker</code>	Same image, 4096 MB, 900s timeout, <code>LAMBDA_ROLE=worker</code> . Loads the full price store on cold start; runs all scheduled jobs.
API Gateway	HTTP API (v2)	Custom domain <code>api.rigacap.com</code> , behind CloudFront + WAF, HTTP/2, CORS allowlist. 29s hard timeout — long jobs invoke the Worker directly.
ECR Repository	<code>rigacap-prod-api</code>	Mutable tags, scan-on-push, retains the last five images for rollback.
Step Functions	<code>rigacap-prod-walk-forward</code>	Fans walk-forward periods out across the Worker Lambda for parallel simulation.

Edge & protection

RESOURCE	IDENTIFIER	PURPOSE
CloudFront (API)	<code>api.rigacap.com</code>	Edge CDN in front of API Gateway. Injects an <code>X-Origin-Verify</code> custom header on every origin request so direct execute-api hits are rejected by the FastAPI middleware.
CloudFront (web)	<code>rigacap.com</code> · <code>www.rigacap.com</code>	Static SPA distribution; SPA-routing 404 fallback to <code>/index.html</code> .
WAF Web ACL	<code>aws_wafv2_web_acl.cloudfront</code>	Attached to the API CloudFront distribution. Managed rule sets plus a rate limit on auth endpoints.

Storage & cache

STORE	ACCESS	PURPOSE
<code>rigacap-prod-frontend-*</code> (S3)	Public via CloudFront	React SPA static assets (1–2 MB total).
<code>rigacap-prod-price-data-*</code> (S3)	Private (Lambda IAM only)	Price store (raw per-symbol bars, split-adjusted at read; <code>all_data.parquet</code> base), dashboard JSON, chart cards, social images, circuit-breaker state, Lambda env backups.
<code>rigacap-prod-terraform-state-*</code> (S3)	Private (operator IAM)	Remote Terraform state with versioning enabled.
ElastiCache Redis	Private (VPC / Lambda)	Hot read cache for dashboard and status payloads; reduces RDS pressure under load.

Database

RESOURCE	CONFIGURATION
RDS PostgreSQL 15	<code>db.t3.micro</code> , 20 GB gp2, deletion protection on, asyncpg driver.
Connection pool	Pool size 5, max overflow 10, connect timeout 3s, query timeout 5s. Lambda connects lazily on first request to stay inside the 10s init window.
Migrations	Applied via the <code>run_migration</code> Worker event, ahead of any model code that depends on the new column (migration-first pattern, §11).

Scheduled events (EventBridge)

The platform's heartbeat is a set of EventBridge rules pointing at the Worker Lambda. Each rule passes a payload that selects a handler. The table below covers the load-bearing rules.

RULE	SCHEDULE (ET)	DESCRIPTION
Market scanner	4:00 PM, Mon–Fri	Full daily scan, two-tier signal generation, dashboard JSON export. Fires after the close so EOD bars have settled.
Double-signal alerts	5:00 PM, Mon–Fri	Multi-confirmation alert email.
Daily digest	6:00 PM, Mon–Fri	Subscriber-facing daily summary email (freshness-gated).
Newsletter draft	Sat 10:00 AM	Generates the locked weekly draft for admin review.
Newsletter publish	Sun 10:00 AM	Publishes the approved newsletter to subscribers.
Nightly walk-forward	Tue–Sat overnight	Walk-forward simulation; feeds the social-content draft queue.

RULE	SCHEDULE (ET)	DESCRIPTION
AI social content	Tue–Sat overnight	Generates social drafts from real trade events.
Intraday monitor	every 5 min, market hours	High-water-mark tracking, regime exits, sell-alert dispatch.
Pipeline health	7:00 AM daily	Full pipeline-health digest to the operator.
Onboarding drip	10:00 AM daily	Five-step new-subscriber sequence over eight days.
Publish posts	every 15 min	Auto-publishes approved social posts whose <code>scheduled_for</code> has passed.
Post notifications	hourly	T-24h and T-1h admin notifications for upcoming posts.
Price-store rebuild	Sat overnight	Weekly full store rebuild — catches new universe symbols.
Nightly data hygiene	6:30 PM, Mon–Fri	Asset-ID integrity, Alpaca corp-actions poll, split auto-refetch.
Meta token refresh	Sun overnight	Weekly long-lived token refresh for Instagram and Threads; admin email on failure.
Lambda warmer	every 5 min	Keeps the Worker Lambda warm.

Edge in front of *everything*.

Both the static SPA and the API live behind CloudFront. The web distribution handles SPA routing; the API distribution attaches the WAF and injects the origin-verify header that closes the direct-execute-api bypass. ACM provides the certificates; Route53 provides the DNS.

CloudFront — web distribution

SETTING	VALUE
Domains	<code>rigacap.com</code> , <code>www.rigacap.com</code>
Origin	S3 website endpoint (HTTP-only)
Protocol	Redirect HTTP → HTTPS
Price class	PriceClass_100 (US, Canada, Europe)
Default TTL	1 hour, max 1 day
SPA routing	Custom error: 404 → <code>/index.html</code> (200)
SSL certificate	ACM: <code>rigacap.com</code> + <code>*.rigacap.com</code> (DNS-validated)

CloudFront — API distribution

SETTING	VALUE
Domain	<code>api.rigacap.com</code>
Origin	API Gateway HTTP API

SETTING	VALUE
WAF	<code>aws_wafv2_web_acl.cloudfront</code> attached
Origin custom header	<code>X-Origin-Verify: <secret></code> — validated by FastAPI middleware
Caching	Disabled for authenticated routes; bypass on Authorization header

Route53 DNS

RECORD	TYPE	TARGET
<code>rigacap.com</code>	A (alias)	Web CloudFront distribution
<code>www.rigacap.com</code>	A (alias)	Web CloudFront distribution
<code>api.rigacap.com</code>	A (alias)	API CloudFront distribution
SPF / DKIM / DMARC	TXT	Email authentication for SES deliverability

PostgreSQL 15, async, *conservative*.

The application talks to PostgreSQL 15 over the async `asyncpg` driver via SQLAlchemy 2.0 async sessions. Most tables use auto-increment integer primary keys; the user identity tables use UUIDs. Schema migrations run through a dedicated `run_migration` Lambda event so the migration can land before any model code that depends on it.

Core market data

stock_data

Daily OHLCV with computed indicators. One row per symbol per trading day.

<code>id</code> INTEGER PK	<code>symbol</code> VARCHAR(10) IDX
<code>date</code> DATETIME IDX	<code>open, high, low, close</code> FLOAT
<code>volume</code> FLOAT	<code>ma_50, ma_200</code> FLOAT

signals

Trading signals from the scanner. Lifecycle: active → executed / expired.

<code>id</code> INTEGER PK	<code>symbol</code> VARCHAR(10) IDX
<code>signal_type</code> VARCHAR(10) BUY/SELL	<code>tier</code> VARCHAR(10) preserver/maximizer
<code>price, composite_score</code> FLOAT	<code>volume, volume_ratio</code> FLOAT
<code>stop_loss</code> FLOAT	<code>is_strong</code> BOOLEAN
<code>status</code> VARCHAR(20)	<code>created_at, expires_at</code> DATETIME

positions · trades

Open / closed portfolio positions with trailing-stop state, and closed trade records with realized P&L.

<code>user_id</code> UUID FK→users	<code>symbol</code> VARCHAR(10) IDX
<code>entry_date, entry_price</code> DATETIME, FLOAT	<code>shares, stop_loss</code> FLOAT
<code>highest_price</code> FLOAT	<code>exit_date, exit_price</code> DATETIME, FLOAT
<code>pnl, pnl_pct</code> FLOAT	<code>exit_reason, status</code> VARCHAR

Identity, subscriptions, and *simulation*.

Authentication and subscriptions

users

Multi-provider OAuth-aware accounts. UUID primary keys.

id	UUID	PK	email	VARCHAR(255)	UNIQUE	IDX
password_hash	VARCHAR(255)	NULLABLE	name, role	VARCHAR		
google_id, apple_id	VARCHAR(255)	UNIQUE	stripe_customer_id	VARCHAR(255)	UNIQUE	
is_active	BOOLEAN		created_at, last_login	DATETIME		

subscriptions

Stripe-managed subscription state, including the Preserver base and the Maximizer add-on. Status: trial / active / canceled / expired / past_due.

id	UUID	PK	user_id	UUID	FK→users	UNIQUE
status	VARCHAR(20)		tier	preserver / maximizer		
stripe_subscription_id	VARCHAR(255)		stripe_price_id	VARCHAR(255)		
trial_start, trial_end	DATETIME		current_period_start/end	DATETIME		
cancel_at_period_end	BOOLEAN					

referrals

Stripe-coupon-backed give-month / get-month program. Eight-character codes.

id	INTEGER	PK	referrer_user_id	UUID	FK→users	
code	CHAR(8)	UNIQUE	stripe_coupon_id	VARCHAR(50)		
redeemed_user_id	UUID	FK→users	redeemed_at, rewarded_at	DATETIME		

Strategy management

strategy_definitions

Strategy configurations backing the Preserver and Maximizer tiers, plus retained legacy definitions for historical comparison.

id	INTEGER PK	name	VARCHAR(100) UNIQUE
strategy_type	VARCHAR(50)	parameters	TEXT (JSON)
is_active	BOOLEAN IDX	source	VARCHAR(50) manual/ai

strategy_evaluations · strategy_switch_history

Strategy scoring and an audit trail of every parameter switch with before/after scores and trigger.

strategy_id	FK→strategy_definitions	total_return_pct	FLOAT
sharpe_ratio	FLOAT	max_drawdown_pct	FLOAT
recommendation_score	FLOAT 0-100	trigger	manual / auto_scheduled
score_before / after	FLOAT		

Simulation, intelligence, and social

walk_forward_simulations

Walk-forward runs with aggregate results. Trades stored as JSON to fit inside the Step Functions state limit.

id	INTEGER PK	start_date, end_date	DATETIME
frequency	VARCHAR(20)	total_return_pct	FLOAT
sharpe_ratio	FLOAT	max_drawdown_pct	FLOAT
trades_json, equity_curve_json	TEXT (JSON)	status	pending/completed/failed
is_daily_cache	BOOLEAN IDX		

model_positions

Live, walk-forward, and signal-track-record positions. Tracks signal replay and AI autopsy.

id INTEGER PK	portfolio_type VARCHAR(20) IDX
symbol VARCHAR(10) IDX	entry_date, entry_price DATETIME, FLOAT
shares, cost_basis FLOAT	highest_price, exit_price FLOAT
exit_reason, status VARCHAR	signal_data_json, autopsy_json TEXT (JSON)

regime_forecast_snapshots · regime_history

Daily regime forecast with the seven-regime probability vector, plus weekly regime classifications used for chart-band overlays.

snapshot_date DATETIME UNIQUE IDX	current_regime VARCHAR(30)
probabilities_json TEXT (JSON)	outlook stable/deteriorating/improving
spy_close, vix_close FLOAT	week_start, regime DATE, VARCHAR(30)

social_posts

AI-drafted social content with the admin approval workflow. Status: draft → approved → scheduled → posted / canceled.

id INTEGER PK	post_type trade_result / missed_opp / recap / contextual_reply
platform twitter / instagram / threads	status, text_content VARCHAR, TEXT
image_s3_key VARCHAR(500)	scheduled_for, posted_at DATETIME
ai_generated, ai_model BOOLEAN, VARCHAR	ai_prompt_hash VARCHAR
news_context_json, source_trade_json TEXT (JSON)	reply_to_* VARCHAR (platform reply IDs)

symbol_metadata · symbol_metadata_events · push_tokens

Layer-2 data hygiene (one row per symbol with Alpaca asset UUID + status, plus a corporate-action audit log) and Expo push tokens for the mobile app.

symbol VARCHAR(10) UNIQUE IDX	asset_id UUID
status active / inactive / quarantined	event_type split / merger / delisting / reuse
token, platform VARCHAR (ios/android)	detected_at, created_at DATETIME

Two sources, one *truth*.

The platform ingests daily market data from two providers with automatic failover, adjusts it once at read time, and exposes a single read interface everywhere downstream — so the production scanner and the walk-forward simulator see exactly the same bars, to the penny.

Dual-source ingestion

A `DualSourceProvider` wraps Alpaca and yfinance with health tracking and auto-failover. The split arose from a stale-data incident: relying on a single vendor left a two-day gap that reached subscribers. The provider now routes each use case to the source that is most reliable for it, and falls back automatically when a source degrades.

USE CASE	PRIMARY	FALLBACK	REASON
Daily EOD bars	Alpaca	yfinance	Faster and more reliable for bulk historical pulls.
Live intraday quotes	yfinance	Alpaca	Alpaca free tier is IEX-only; yfinance covers all exchanges.
Index data (^VIX, ^GSPC)	yfinance	—	Alpaca does not serve index symbols.
Sector ETFs	Alpaca	yfinance	Normal tickers, Alpaca-compatible.

If more than ten percent of symbols fail during a daily scan, the pipeline automatically retries the failed set against the yfinance fallback. Symbol normalization reconciles the

hyphen/dot mismatch between providers (`BRK-B` ↔ `BRK.B`). Splits are pulled on every ingest so the cached history stays split-adjusted, preventing the unadjusted-split artifacts that surface after large stock splits.

Storage layers

Per-symbol price store

`S3 · ADJUST-AT-READ`

Raw daily bars persisted per symbol and split-adjusted at read time, so the same canonical history drives both the live scanner and the walk-forward simulator. Refreshed daily; a weekly full rebuild incorporates new universe symbols.

Parquet columnar store

`PRICES/ALL_DATA.PARQUET`

Columnar, compressed, schema-enforced via pyarrow. Enables partial reads (one symbol without loading the full blob) and DuckDB SQL — the change that took the Worker's scan footprint from OOM-risk down to comfortably inside its ceiling.

CDN dashboard JSON

`DASHBOARD/LATEST.JSON`

Pre-computed at scan time and fronted by CloudFront with a one-hour TTL. The dashboard, the daily digest, the sell-alert dispatcher, and the intraday monitor all read from this single JSON so emails and the UI never disagree on the day's signals.

Data-integrity filters

`SCANNER DATA-QUALITY GATE`

Rejects extreme price ratios, single-day moves above a log-return threshold, and multi-day gaps — the ticker-reuse signature. Catches unadjusted splits, stitched histories, and delisting artifacts in real time.

Freshness gate

Every subscriber-facing communication job (daily digest, sell alerts, double-signal alerts) validates that the daily scan ran successfully *today* before it sends. If the data is stale, emails are held and the operator is alerted rather than shipping a subscriber a signal computed on yesterday's prices. The dashboard exposes the same state

publicly through `/api/market-data-status`, which drives a staleness banner in the UI.

Layer-2 data hygiene

Each weekday evening the `nightly_data_hygiene` rule runs a protection pass against silent data drift:

1. **Asset-ID verification.** Compares each symbol's stored Alpaca asset UUID to the current value; any change is treated as ticker reuse — auto-quarantine plus admin alert.
2. **Corp-actions poll.** Queries the Alpaca corporate-actions API for splits, dividends, spin-offs, mergers, and delistings.
3. **Split auto-refetch.** For each detected split, fetches a clean split-adjusted history and overwrites the cache.
4. **Store re-export.** The updated cache is re-exported so the read stores never diverge.
5. **Post-actions diagnose.** A DuckDB scan over the fresh parquet flags anomalies above threshold.
6. **Admin digest.** Auto-remediated items report as INFO; only true anomalies escalate.

One scan, two *reads*.

The backend is a set of singleton service classes instantiated at module load and imported across handlers. Mangum translates ASGI calls into Lambda invocations on the API path; the Worker Lambda dispatches EventBridge payloads through a handler map.

How the two tiers are served

RigaCap is a two-tier product — **Preserver** (protect capital) and **Maximizer** (grow / breakout). Both are produced by the same regime-adaptive momentum engine over the same universe on the same daily scan; they differ in their exposure and risk posture, not in a separate data path. A single scan computes both reads and writes both into the dashboard JSON, tagged by tier. At request time, the subscription tier on a user's account gates which read they see: Preserver is the base, and Maximizer is a paid add-on unlocked through Stripe. This keeps a subscriber's live experience faithful to the tier they pay for, and keeps both tiers reconciled to the same underlying walk-forward record.

Marketing-parity discipline. The engine warm-starts from full history on every run — production has years of bars, and starting cold would misprice the risk controls. Any figure a subscriber sees, and any exit parameter stated in the UI or an email, must be the live tier value, so the published record is actually realizable by a subscriber who mirrors it.

Risk machinery (public-safe description)

Seven-regime market classifier

STRONG_BULL / WEAK_BULL / ROTATING_BULL / RANGE_BOUND / WEAK_BEAR /
PANIC_CRASH / RECOVERY

Daily classification from a broad-market indicator stack (long-term trend, breadth, volatility). The classified regime modulates exposure, sizing, and exit thresholds — the mechanism behind “momentum, built around the drawdown.” Snapshots persist for heatmap visualization and accuracy tracking.

Cascade Guard (circuit breaker)

STATE PERSISTED TO S3 PER PORTFOLIO TYPE

When many positions hit their trailing stops on the same day — the cascading-stress signature — the guard pauses new entries for a cooling window. State persists to S3 so the breaker survives Lambda cold starts. Externally this is “Cascade Guard”; internally it is the circuit breaker.

Service inventory

SERVICE	RESPONSIBILITY
ScannerService	Daily market scan, two-tier signal generation, in-memory cache management.
BacktesterService	Historical portfolio simulation and performance metrics.
WalkForwardService	Step Functions orchestration, period-by-period evaluation, AI optimization.
MarketRegimeService	Seven-regime classification with daily snapshot persistence.
CircuitBreakerService	Cascade Guard implementation with S3-persisted state.
DualSourceProvider	Alpaca primary + yfinance fallback with health tracking and auto-failover.
DataExportService	S3 export/import, CDN pre-computation, parquet persistence, DuckDB queries.

SERVICE	RESPONSIBILITY
SymbolMetadataService	Layer-2 hygiene: asset-ID checks, corp-actions handling, quarantine list.
EmailService	Subscriber emails: welcome, daily digest, sell alerts, password reset, onboarding drip — all freshness-gated.
NewsletterService	Weekly newsletter: locked Saturday draft, Sunday publish, archive teasers.
ModelPortfolioService	Live, walk-forward, and signal-track-record tracking; what-if calculator.
AIContentService	Claude API client for social drafts, autopsies, and editorial copy; template fallback if the API is unavailable.
SocialContentService	Template-based post generation; safety net for the AI service.
PostSchedulerService	Spreads drafts across optimal windows; T-24h and T-1h notifications; one-click cancel.
SocialPostingService	Multi-platform publish: X, Threads, and Instagram.
ReplyScannerService	Contextual reply scanning across X and Threads.
ChartCardGenerator	1080×1350 (4:5) editorial chart cards for Instagram and social.
TradeAutopsyService	Claude-powered post-mortem analysis on closed trades.
RegimeForecastService	Daily regime forecast storage and accuracy tracking.
StockUniverseService	Universe management, sector classification, ETF filtering.
StrategyGeneratorService	Optuna-based parameter optimization.
HealthMonitorService	Pipeline health across data freshness, worker performance, database, signals, and infrastructure.

SERVICE

RESPONSIBILITY

ReferralService

Stripe-coupon-backed give-month / get-month program.

PushNotificationService

Expo push notifications for the mobile app.

Lambda cold-start mitigation. Database connections initialize lazily on first request because the Lambda init phase has a ten-second hard ceiling. An EventBridge rule pings the health route every five minutes to keep the Worker warm; post-deploy, a warm-up route preloads S3 data into memory.

120+ endpoints, *auth-first*.

The FastAPI surface exceeds 120 endpoints across authentication, signals, billing, dashboard, social, admin, and public marketing routes. Counts below are approximate; the relevant router file is the source of truth. Anything that returns signal or portfolio data sits behind `require_valid_subscription` or the admin guard.

Authentication — `/auth`

METHOD	PATH	DESCRIPTION
POST	/auth/register	Email/password signup with Turnstile verification.
POST	/auth/login	Email/password login; returns JWT access + refresh tokens.
POST	/auth/refresh	Refresh access token.
GET	/auth/me	Current user profile, including tier.
POST	/auth/google · /auth/apple	Google OAuth and Apple Sign In.
POST	/auth/forgot-password · /auth/reset-password	Password reset flow with secure token.
POST	/auth/unsubscribe	One-click email unsubscribe (RFC 8058).
GET	/auth/referral	Get / create referral code and stats.

Signals — `/signals`

METHOD	PATH	AUTH
GET	/signals/dashboard	Subscriber — comprehensive dashboard data, filtered to the account's tier.
GET	/signals/latest	Subscriber.
GET	/signals/watchlist	Subscriber — approaching-trigger names.
GET	/signals/double-signals	Subscriber — multi-confirmation.
GET	/signals/symbol/{symbol}	Subscriber.
POST	/signals/backfill	Admin.

Billing — `/billing`

METHOD	PATH	DESCRIPTION
POST	/billing/create-checkout	Stripe Checkout for the selected tier; trial with credit card required.
POST	/billing/portal	Stripe Customer Portal session (upgrade / add Maximizer / cancel).
GET	/billing/subscription	Current subscription status and tier.
POST	/billing/webhook	Stripe webhook receiver, signature-verified.

Public & dashboard — `/api`

METHOD	PATH	DESCRIPTION
GET	/api/market-data-status	Public freshness signal for the dashboard staleness banner.

METHOD	PATH	DESCRIPTION
GET	/api/track-record	Public: walk-forward and live data backing the <code>/track-record</code> page.
GET	/api/newsletter/archive	Public: archive of past newsletter issues.
GET	/api/warmup	Post-deploy warm-up route — preloads dashboard JSON.

Social — `/social`

METHOD	PATH	DESCRIPTION
GET	/social/posts · /social/posts/{id}	List and inspect posts — filter by status, type, platform.
POST	/social/posts/{id}/approve · /reject	Approve for publishing or reject with reason.
POST	/social/posts/{id}/schedule · /cancel	Schedule for a future window or cancel.
POST	/social/posts/{id}/regenerate-ai	Re-draft via the Claude API.
GET	/social/posts/{id}/approve-email · /cancel-email	One-click Kill / approve via JWT-signed link (no login required).
POST	/social/generate-chart/{id}	Generate the editorial chart-card image.

Admin — `/admin`

Admin endpoints require both `role=admin` and an `ADMIN_EMAILS` allowlist match — double enforcement so a mistakenly-promoted user cannot reach operator surfaces. Categories include strategy management, user management, regime analysis, walk-

forward simulation, data debugging, model portfolio, trade autopsies, and the what-if calculator.

React, Vite, Tailwind — *editorial* register.

The web client is a single-page React 18 application built with Vite and styled with Tailwind CSS in the editorial brand palette — paper, ink, and claret — with Fraunces and IBM Plex as the typefaces. The mobile app is a separate Expo project that consumes the same API surface for sell alerts and signal events.

Component inventory (representative)

COMPONENT	RESPONSIBILITY
App.jsx	Main dashboard: portfolio, tier-filtered signals, charts, auth, admin time-travel.
LandingPage.jsx	Marketing surface: hero, two-tier pricing, FAQ, social proof.
TrackRecordPage.jsx	Public <code>/track-record</code> : continuous walk-forward results plus the live record.
WalkForwardSimulator.jsx	Configure and run simulations; equity curves, trade analysis.
SocialTab.jsx	Post queue, AI generation, scheduling, chart cards, publishing.
AdminDashboard.jsx	Strategy and user management, model portfolio, regime forecast, what-if calculator, trade autopsies.
NewsletterArchive.jsx	Newsletter archive teasers with headline and breadcrumb lines.
LoginModal.jsx	Multi-provider auth with Turnstile.
SubscriptionBanner.jsx	Trial countdown, upgrade / add-Maximizer CTA.

CookieConsent.jsxGDPR cookie consent gating GA4.

Frontend libraries

- React 18 with hooks; functional components throughout.
- Vite 5 build tool; fast HMR in dev, single-bundle production output.
- Tailwind CSS 3 with the claret/paper brand tokens in `tailwind.config.js`.
- Recharts for dashboard charts and equity curves.
- Axios with a JWT request interceptor and a 401 retry hook.
- React Router v6 for SPA routing; Lucide React icons.
- react-hot-toast, date-fns, clsx for UX plumbing.
- An ErrorBoundary wraps the entire app; GA4 is loaded only after cookie consent.

Real trades in, moderated posts *out*.

The social engine turns real walk-forward and live results into editorial posts — drafted by Claude, approved by a human in one click, and published on a fixed cadence across three platforms. Nothing reaches an audience without an operator's explicit approval.

The pipeline

1. **Draft.** The nightly walk-forward and live-trade events feed `AIContentService`, which prompts the Claude API to write posts grounded in the actual result — result-and-thesis first, methodology hygiene left out of customer copy. A template service is the fallback if the API is unavailable.
2. **Schedule.** `PostSchedulerService` spreads approved drafts across optimal posting windows on a Monday / Wednesday / Friday autopost cadence, capped per day, and issues T-24h and T-1h admin notifications.
3. **Approve or Kill.** Each notification email carries a JWT-signed one-click Kill or approve link, and an edit path back into the queue — no login required, scoped to a single post ID.
4. **Publish.** A cron every fifteen minutes publishes any approved post whose `scheduled_for` has passed.

Multi-platform publishing

PLATFORM	FORMAT	NOTES
X	Text post	Twitter API v2. First-party posts publish via the API.

PLATFORM	FORMAT	NOTES
Threads	Text post	Meta graph API; long-lived token refreshed weekly.
Instagram	Editorial quote card	1080×1350 chart / quote card rendered in the claret/paper brand, published via the graph API.

Contextual replies (deep-link workaround)

A reply scanner surfaces on-thesis conversations across X and Threads and drafts a contextual reply for approval. Because the X free tier cannot programmatically post replies where the account was not mentioned, the approval email **deep-links to the platform's pre-filled composer** (an `in_reply_to` intent URL) rather than posting through the API. The operator taps once and sends. First-party posts still publish through the API directly — only replies use the deep-link path.

Human in the loop, by design. Every AI draft — post or reply — is gated behind explicit approval. The Kill link makes the default action “do nothing” safe: an unapproved post simply never ships.

Defense in *depth*, not in marketing.

Every signal endpoint sits behind a subscription guard. Every admin endpoint sits behind two. The price-data S3 bucket has all public access blocked. The raw execute-api URL is not reachable directly. None of this is novel; all of it is enforced and re-verified on a fixed cadence.

Authentication flow

```
# Subscriber login
Email / Google / Apple → Turnstile (bot guard)
                        → JWT issue (HS256, short-lived access)
                        → Bearer header on every request
                        → Role check: admin / user
                        → Subscription + tier check
```

Controls

LAYER	IMPLEMENTATION
Password hashing	bcrypt via passlib.
JWT tokens	HS256, short-lived access / longer-lived refresh, python-jose.
OAuth validation	Google ID-token signature against published keys; Apple JWKS with cache.
Bot protection	Cloudflare Turnstile on registration.

LAYER	IMPLEMENTATION
CORS	Allowlist: <code>rigacap.com</code> , <code>www.rigacap.com</code> , plus localhost dev.
Admin guard	Role plus <code>ADMIN_EMAILS</code> allowlist — double enforcement.
Subscription gate	<code>require_valid_subscription</code> on every signal and portfolio endpoint; tier gates Maximizer content.
Payment security	Stripe Checkout with webhook signature verification; RigaCap stores no card data.
S3 access	Price-data bucket: all public access blocked, IAM-only. No signal data on the CDN or a public path.
HTTPS	ACM certificates everywhere; CloudFront redirects HTTP → HTTPS.
WAF	Web ACL on the API distribution with managed rule sets and a rate limit on auth.
Origin verify	CloudFront injects <code>X-Origin-Verify</code> on every API origin request; middleware compares it with a constant-time check. Direct hits to the raw execute-api URL return 403.
Signed email links	JWT-signed one-click approve / Kill / unsubscribe URLs, each scoped to a single resource.

Review cadence

A security audit runs every two to three weeks. The reviewer verifies that every signal, dashboard, and portfolio endpoint requires auth; that the price-data bucket remains private; that no signal data leaks through the CDN or a public S3 path; that the CORS allowlist matches only the production origins plus dev; and that any endpoints added since the last review carry proper guards. No secrets ship in the frontend bundle or in git history.

Push to main, the rest is *plumbing*.

Every change ships through GitHub Actions on push to `main`. The pipeline tests, builds the frontend, builds the Lambda container image, deploys to ECR, updates both Lambdas to the same image, and warms the API. Manual deploys are reserved for CI outages.

```
# Trigger: push to main (deploy) OR PR to main (test only)

Stage 1: Test
  Backend  → Python 3.11 → pip install → pytest -v
  Frontend → Node 18     → npm ci       → npm run lint && npm test

Stage 2: Deploy (only on main push)
  1. npm run build → S3 sync → CloudFront invalidation
  2. docker buildx --platform linux/amd64 --provenance=false --sbom=false
  3. ECR push → Lambda update-function-code on API + Worker
  4. Post-deploy: GET /api/warmup (verify cold start healthy)
```

Lambda build requirement. Docker BuildKit creates multi-platform manifests with attestations that Lambda rejects. All builds use `--provenance=false --sbom=false` to produce the single-platform `linux/amd64` image Lambda requires.

The environment-clobber landmine, disarmed

Two operational footguns have bitten this platform and are now guarded against in the tooling itself:

- Never `update-function-configuration --environment`. That flag *replaces the entire* environment block, which would wipe the database URL, Stripe keys, and

JWT secret in one call. Cold starts are forced with `update-function-code` against the current image instead.

- Terraform cannot clobber live env either. Both Lambdas carry `lifecycle { ignore_changes = [image_uri, environment] }`. The live functions accumulate keys through the container deploy path; without the guard, a blind `apply` would drop keys and revert values — a full outage. Plans still get reviewed before apply.

Disaster recovery & backups

ASSET	PROTECTION
Lambda environment	Full env-var set backed up to S3 (and a local JSON) before any change, so a clobber is a one-command restore.
Database	RDS automated backups; deletion protection enabled on the instance.
Container images	ECR retains the last five images — rollback is redeploying a prior tag.
Terraform state	Versioned S3 backend with DynamoDB locking.
Price history	Durable in S3; a weekly full rebuild reconstructs the store from source if needed.

Migration discipline

SQLAlchemy auto-includes every model column in SELECT queries, so a column that exists in the model but not yet in the database breaks every query against that table — including auth. The platform follows a migration-first pattern: deploy the migration SQL first via the `run_migration` Lambda event, verify the columns exist, then deploy the model change in a second commit. Schema changes never ship during business hours.

Terraform & account hygiene

Infrastructure changes flow through Terraform against account `149218244179`, always run with the dedicated `AWS_PROFILE=rigacap` — the default profile points at a

different account and is explicitly never used for production work.

Boring numbers, *watched*.

Monitoring

COMPONENT	METHOD	DETAILS
Lambda logs	CloudWatch Logs	Structured Python logging.
CloudWatch alarms	SNS → email	Eight alarms: Lambda errors / throttles / duration; API 5xx and 4xx; RDS CPU / storage / connections.
Pipeline health	HealthMonitorService	Multi-category checks across data freshness, worker performance, database, signals, and infrastructure.
Data-fetch tracking	S3 metadata JSON	Source used (Alpaca / yfinance), settlement result, fetch duration, fallback status.
Freshness banner	/api/market-data-status	Public staleness signal — blue during the processing window, amber if data is late.

Performance characteristics

OPERATION	LATENCY	RESOURCE
Dashboard load	< 2 seconds	Pre-computed JSON from CloudFront.
Daily signal scan	seconds, not minutes	Worker Lambda, ~4,000 symbols, dual-source.
Walk-forward (multi-year)	10–30 minutes	Step Functions fan-out across the Worker Lambda.

OPERATION	LATENCY	RESOURCE
AI content generation	2–5 seconds	Claude API.
Chart-card generation	1–3 seconds	matplotlib on the Worker Lambda.

Resource footprint

1 + 4 GB API + WORKER MEMORY	~4,000 SYMBOLS SCANNED DAILY	20 GB RDS STORAGE
---	---	--

Manual Lambda invocations always target the Worker so the API surface stays available for subscribers; long-running jobs (walk-forward simulations, AI optimization) invoke the Worker directly to bypass the 29-second API Gateway timeout.

Dependencies — backend (Python 3.11 on Amazon Linux 2023)

- FastAPI + Mangum (ASGI-to-Lambda), Uvicorn for local dev.
- SQLAlchemy 2.0 async with asyncpg; Pydantic 2 models.
- pandas / numpy for computation; pyarrow + duckdb for the parquet store and ad-hoc SQL.
- alpaca-py and yfinance behind the dual-source provider.
- Optuna for parameter optimization; matplotlib for chart cards.
- boto3 (AWS SDK), Stripe (payments), python-jose + passlib (JWT, bcrypt).
- httpx async client for the Claude API and the social graph APIs.

Dependencies — frontend, mobile & external

- Frontend: React 18 · Vite 5 · TailwindCSS 3 · React Router 6 · Recharts · Axios · Lucide React.
- Mobile: React Native via Expo; Expo push for sell alerts and signal events.

- External APIs: Anthropic Claude (content, autopsies, editorial copy) · X / Threads / Instagram graph APIs · Stripe · Cloudflare Turnstile · Google & Apple OAuth · Amazon SES · Expo Push.

This document is confidential. Infrastructure details should not be shared outside the organization. Code-level identifiers are accurate as of the document date and may evolve through normal deployment activity.

RigaCap publishes momentum stock-signal research in two tiers — Preserver and Maximizer. It is a publisher, not an investment adviser: the same signals reach every subscriber at a given tier regardless of their portfolio or situation, and subscribers execute their own trades through their own broker. All performance figures are walk-forward results validated across cross-period holdouts and daily-resolution drawdowns. Past performance does not predict future results.